

Candidate brief

Regulation ingest and tracker

Before getting into the task itself, a few points worth clarifying.

- **Time budget: 2-3 hours.** We want to stress that we do not expect a perfect or polished project in that time. Building this to production standard in house would take us 2-3 days. We are looking to understand your core capabilities and your approach to the problem, not to receive finished software.
- **We strongly encourage the use of AI tools and AI assisted coding.** At Maiven we use AI throughout our development process and suggest you do too. Attempting all of this by hand would likely take far too long. We won't penalise you for choosing not to, but we would encourage you to reach for Claude, Codex, Cursor, or whichever tool you prefer.
- **You should still be able to explain and justify your decisions.** The next step in the process is a technical discussion of this work, in which we will also look at building on top of it. Understanding what you have produced is essential.

Context

Maiven ingests regulatory documents from public sources, normalises them, and makes them searchable and trackable for compliance teams. This exercise is a miniature version of that pipeline, end to end.

Stack

The stack below is a recommendation rather than a requirement, and you are welcome to work with what you are comfortable with. We would ask that you use **Python for the ingest** and **TypeScript for the serve and display** parts, since that reflects how we work.

- Python - ingest
- TypeScript, Next.js (App Router) - API and page
- PostgreSQL

Task 1 - Ingest

Pull recent **Rules** issued by the **Environmental Protection Agency** from the US Federal Register API. The API is public, requires no authentication, and all of its content is in the public domain.

API documentation: <https://www.federalregister.gov/developers/documentation/api/v1>

The documents endpoint: <https://www.federalregister.gov/api/v1/documents.json>

TASK 1 / INGEST (CONTINUED)

Here is an example query you can run with curl to examine the output.

```
curl -s \
  "https://www.federalregister.gov/api/v1/documents.json?"\
  "conditions\[type\]\[\]=RULE&"\
  "conditions\[agencies\]\[\]=environmental-protection-agency&"\
  "per_page=20&fields\[\]=title&fields\[\]=document_number&"\
  "fields\[\]=publication_date&fields\[\]=effective_on&"\
  "fields\[\]=abstract&fields\[\]=agencies&fields\[\]=html_url"
```

Write a script that fetches these documents, normalises them, and stores them. A local Postgres database is the straightforward option, but a structured file format such as CSV or Parquet is equally acceptable if that suits your approach better. Either is fine, provided you can explain the choice.

Basic requirements

1. Re-running the script must not duplicate rows or lose data.

2. A run should ingest 100 documents. That means handling pagination. Note that the count and total_pages fields in the response may not be reliable.

Task 2 - Serve

Create an API endpoint that returns the documents you have stored.

GET /api/documents - returns stored documents, most recently published first.

It should:

- Filter by a date range on publication_date
- Support a simple case-insensitive text search
- Return 20 documents at a time, with a way to fetch the next 20

Task 3 - Display

A single page listing the documents, with the filter and search wired up and a "load more" control. Nothing fancy, just something that shows us the data you have ingested.

Task 4 - Tests

Cover the text cleaning and the re-run behaviour of the ingest. You don't need exhaustive coverage; a couple of meaningful tests is fine.

Task 5 - README

Please include:

- How to run it, including any requirements
- What you would do differently with more time

Submitting

Please send a zip of your code to jess@wearekusudi.org

AND ideally a git repository: please give access to the GitHub user:

joshjbayne@gmail.com